

NumPy

NumPy is a Python library that provides efficient implementations of many functions commonly used in scientific programming. NumPy is probably the most widely used library in the scientific Python stack because it provides one crucial data type--multi-dimensional arrays--that allow us to efficiently manipulate large datasets. NumPy arrays are used and supported by nearly every Python analysis and visualization library, making it the de-facto standard for binding these tools together.

What is an array?

Arrays are a very useful concept shared by many programming languages. Arrays are:

1. Containers that hold many items, all of the same type.
2. Represented by a rectangular structure. Arrays may have any number of dimensions, but each dimension (axis) of the array has a fixed length.

For example:

- A 1D array of 10M samples recorded from an electrode

0.0531	0.0547	0.0522	0.0525	0.0536	0.0531	. . .	0.0530
--------	--------	--------	--------	--------	--------	-------	--------

- A 5x5 (2D) matrix

1	0	0	0	0
0	1	0	0	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	1

- A 500x500 pixel image with 3 color channels (3D)

 No description has been provided for this image

More on arrays

[The N-dimensional array \(ndarray\)](#)

Why are arrays useful?

- They are the standard data structure used by most analysis / visualization tools
- They are fast
- They use memory efficiently
- Used correctly, they can make your code more concise

In principle, we could create multi-dimensional array-like structures simply by nesting Python lists together. In practice, this approach is tedious and inefficient.

As an example, let's try constructing a 2D array out of nested Python lists, and compare this approach to using NumPy arrays.

I will build an "array" of random numbers that is 200 columns x 50k rows:

```
In [1]: import random

rows = 100000
cols = 200

# Create a list that is 100,000 rows long, where each item in the list
# is a nested list of 200 random values.
awkward_array = []
for i in range(rows):
    row = []
    for j in range(cols):
        row.append(int(10*random.random()))
    awkward_array.append(row)

# display a small subset of this "array"
[row[:15] for row in awkward_array[:10]]
```

```
Out[1]: [[7, 7, 7, 1, 6, 3, 5, 8, 0, 6, 1, 8, 4, 2, 7],
         [2, 8, 9, 9, 1, 4, 3, 6, 3, 8, 6, 4, 2, 7, 4],
         [9, 4, 6, 4, 2, 1, 0, 1, 9, 2, 0, 3, 6, 2, 2],
         [5, 2, 9, 8, 9, 5, 4, 9, 6, 8, 3, 1, 0, 3, 4],
         [3, 3, 8, 1, 0, 9, 6, 0, 7, 8, 7, 0, 6, 2, 0],
         [5, 8, 0, 1, 6, 2, 5, 5, 7, 7, 6, 8, 2, 2, 3],
         [5, 8, 5, 0, 7, 2, 5, 2, 3, 4, 9, 5, 4, 6, 0],
         [6, 3, 7, 6, 7, 7, 2, 0, 4, 7, 9, 7, 1, 1, 0],
         [2, 0, 8, 4, 7, 2, 6, 7, 1, 5, 2, 7, 4, 0, 0],
         [1, 3, 5, 3, 3, 8, 0, 7, 8, 9, 8, 7, 3, 2, 0]]
```

Now I want to calculate the mean value of every column:

```
In [2]: # For each column, collect all values in the column and then compute the average
meanvals = []
for j in range(cols):
    col = [row[j] for row in awkward_array]
    meanvals.append(sum(col) / float(rows))
```

```
print(meanvals[:5])
```

```
[4.5087, 4.5019, 4.50349, 4.48935, 4.49116]
```

This code works, but it is:

- Slow - Python lists can contain mixed data types. One consequence is that when we add up values in a column, Python has to check **each** value to make sure it is a numerical type that can be added to the running total. If the array is large, that overhead can add up quickly!
- Memory-inefficient - We should be able to represent this data set using `rows * cols = 20M numbers`, plus some small overhead. Python lists have about 1kB of memory overhead **per list**, which again can add up quickly. If your data just barely fits in memory, this will ruin your day. Python lists can also vary in length, which means their contents are not necessary contiguous. This can result in slower times for accessing data.
- Verbose - this very simple task can be written much more compactly with the right tools.

NumPy to the rescue

The code we just ran can be expressed much more concisely using a numpy array:

```
In [3]: import numpy as np
```

```
In [4]: data = np.random.random(size=(rows, cols))
meanvals = data.mean(axis=0)
print(meanvals[:5])
```

```
[0.4991632  0.49904239 0.50038601 0.50038795 0.50173143]
```

These three lines of code replace the 16 lines that it took to perform the same operation with lists, and it runs in a small fraction of the time.

Documentation:

[np.random](#)

[numpy.mean](#)

Let's explore the array object we just created:

```
In [5]: # numpy arrays have a set number of dimensions:
data.ndim
```

Out[5]: 2

```
In [6]: # each dimension has a specific length
data.shape
```

Out[6]: (100000, 200)

```
In [7]: # The total number of items in this array (200 * 100k) is given by the size
data.size
```

Out[7]: 20000000

```
In [8]: # The array's memory usage is the number of items (200 * 100k)
# times the number of bytes used by each item
data.size * data.itemsize
```

Out[8]: 160000000

This array uses 160MB of memory to represent 20M values. Why do we use 8 bytes for each item?

```
In [9]: # all items in the array share the same data type
data.dtype
```

Out[9]: dtype('float64')

All elements in the array are of the same data type, a 64-bit (8 byte) floating-point number.

This is the default **dtype**, but there are many other options we can pick from. Whether we **care** enough to worry about dtype depends on what we want to do with the array, and whether we are likely to run out of memory. Some of the most commonly used dtypes are listed here:

dtype	bytes	precision	approx. range
float64	8	16	$\pm 10^{308}$
float32	4	7	$\pm 10^{38}$
int64	8	0	$\pm 10^{18}$
int32	4	0	$\pm 10^9$
int16	2	0	$\pm 10^4$
uint64	8	0	0 to 10^{19}
uint32	4	0	0 to 10^9
uint16	2	0	0 to 10^4

uint8	1	0	0-255
bool	1	0	0-1

Documentation: [Data types](#)

[numpy.ndarray](#)

To convert an array to a different dtype, use the **astype()** method.

```
In [10]: smaller_data = data.astype('float32')

# This should be half the size of the original array
smaller_data.size * smaller_data.itemsize
```

Out[10]: 80000000

Using NumPy Arrays

The rest of this notebook is a crash course on all the ways we use arrays -- creating, modifying, slicing, reshaping, calculating, and visualizing. **Don't try to memorize everything here!** We want you to focus on learning *how arrays are used* and *what they can do*. You should come away from this notebook with the ability to `_strategize_` about solving problems that need arrays. If you forget all the names of the functions we use below, you can always find them again later.

Creating new arrays

NumPy offers several mechanisms for creating new arrays. Let's look at a few of the more common approaches:

1. Nested lists

```
In [11]: # Create an array from nested lists.
list_data = [[1,2], [3,4], [5,6], [7,8]]
arr1 = np.array(list_data)
print(arr1)
```

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

```
In [12]: # list_data is a list of length 4,  
# and each of its items is a nested list of length 2.  
# This corresponds to an array shape (4, 2)  
arr1.shape
```

```
Out[12]: (4, 2)
```

Exercise 4.1: Use `np.array` to create a 3-dimensional array with the shape (2, 4, 3)

```
In [13]: arr = np.array([[[1,2,3], [4,5,6], [7,8,9], [0,1,2]], [[11,22,33], [44,55,66],  
print(arr.shape)  
arr
```

```
(2, 4, 3)
```

```
Out[13]: array([[[ 1,  2,  3],  
                [ 4,  5,  6],  
                [ 7,  8,  9],  
                [ 0,  1,  2]],  
               [[11, 22, 33],  
                [44, 55, 66],  
                [77, 88, 99],  
                [ 0, 11, 22]]])
```

2. zeros()

```
In [14]: # Create an array of a certain shape and dtype, filled with zeros  
arr2 = np.zeros((5, 10), dtype='uint16')  
print(arr2)  
print("shape:", arr2.shape, "dtype:", arr2.dtype)
```

```
[[0 0 0 0 0 0 0 0 0 0]  
 [0 0 0 0 0 0 0 0 0 0]  
 [0 0 0 0 0 0 0 0 0 0]  
 [0 0 0 0 0 0 0 0 0 0]  
 [0 0 0 0 0 0 0 0 0 0]]  
shape: (5, 10) dtype: uint16
```

3. arange()

```
In [15]: # Create a 1D array of consecutive integers  
print(np.arange(5, 11))
```

```
[ 5  6  7  8  9 10]
```

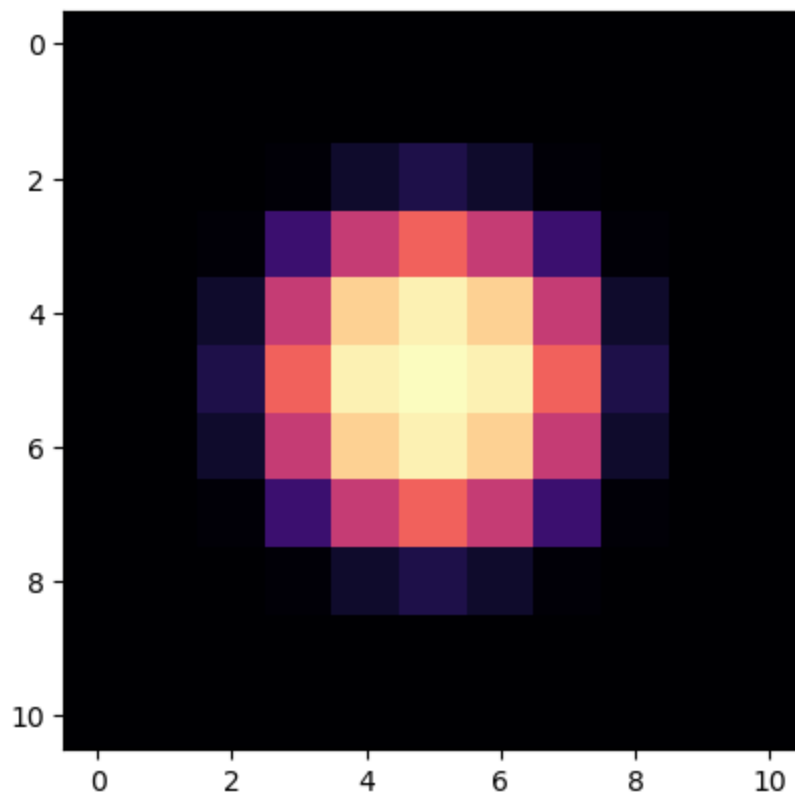
4. fromfunction()

```
In [16]: # Create an array and fill it with values generated by a function.
# For each element in the new array, the function will be called with the el
def gauss(x, y, center=(5, 5), sigma=10):
    dist = ((x-center[0])**2 + (y-center[1])**2) * 0.5
    return 100 * np.exp(-dist**2 / sigma)

gauss_data = np.fromfunction(gauss, (11, 11))
print(gauss_data.astype('int')) # convert to int for easier reading
```

```
[ [ 0  0  0  0  0  0  0  0  0  0  0]
  [ 0  0  0  0  0  0  0  0  0  0  0]
  [ 0  0  0  1  8 13  8  1  0  0  0]
  [ 0  0  1 20 53 67 53 20  1  0  0]
  [ 0  0  8 53 90 97 90 53  8  0  0]
  [ 0  0 13 67 97 100 97 67 13  0  0]
  [ 0  0  8 53 90 97 90 53  8  0  0]
  [ 0  0  1 20 53 67 53 20  1  0  0]
  [ 0  0  0  1  8 13  8  1  0  0  0]
  [ 0  0  0  0  0  0  0  0  0  0  0]
  [ 0  0  0  0  0  0  0  0  0  0  0]]
```

```
In [17]: # Sneak peek: let's visualize that array as an image
# (we will discuss visualization in more detail in the matplotlib notebook)
from matplotlib.pyplot import imshow
imshow(gauss_data, interpolation='none', cmap='magma');
```



Documentation:
[Array creation routines](#)

Accessing single values from arrays

```
In [18]: # Create a small array to play with
arr4 = np.array([[1,2,3,4,5], [6,7,8,9,10], [11,12,13,14,15]])
print(arr4)

[[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]]
```

```
In [19]: # To read a single value from the array, we index the array using [] notation
# and supply a comma-separated list of indices, one per axis
print(arr4[2, 3])
```

14

```
In [20]: # To change one value in the array:
arr4[1, 2] = -1
print(arr4)

[[ 1  2  3  4  5]
 [ 6  7 -1  9 10]
 [11 12 13 14 15]]
```

Exercise 4.2:

In the cell below, we create a small array of random integers.

1. Use `np.zeros` to create a new array with the same shape as `rand_arr`
2. Write a for-loop that sets each item of the new array equal to the corresponding item in `rand_arr` plus one

```
In [21]: rand_arr = np.random.randint(0, 9, size=10)

# Create a new array called rand_arr_plus_one
### Your code here:
rand_arr_plus_one = np.zeros(rand_arr.shape, dtype=int)
for i in range(rand_arr.shape[0]):
    rand_arr_plus_one[i] = rand_arr[i] + 1

### Check your solution:
print(rand_arr)
print(rand_arr_plus_one)

[1 3 4 1 4 6 6 8 1 3]
[2 4 5 2 5 7 7 9 2 4]
```


Documentation

Indexing

Arithmetic with arrays

In the exercise above, you used a nested for-loop to access each element of an array, one at a time. This is fine for small arrays, but for large arrays we would be executing many Python statements, each of which has a significant performance cost.

NumPy provides a way to express this operation (and many others) using a single Python statement:

```
In [22]: # Start with a 1D array of integers 0 through 7
arr5 = np.arange(8)
print(arr5)

# Create a new array by adding 2 to each element of arr5
arr6 = arr5 + 2
print(arr6)
```

```
[0 1 2 3 4 5 6 7]
[2 3 4 5 6 7 8 9]
```

This process--replacing nested for-loops with array operations--is called **vectorization**. It both speeds up our code and makes it easier to read and write.

It is also possible to modify an existing array in-place, rather than creating a new array:

```
In [23]: # subtract 2 directly from each element in arr6; do not create a new array
arr6 -= 2
print(arr6)
```

```
[0 1 2 3 4 5 6 7]
```

Note: most functions in numpy will either **create** and return a new array, or **modify** an existing array in-place. Keep track of which of these is happening in your code, and look for this whenever you read the documentation.

Exercise 4.3:

Create a 1D array that contains the values $(2^x - 1)$ for integer values of x from 0 to 10. For example, the first 3 items are:

$$2^0 - 1 = 0$$

$$2^1 - 1 = 1$$

$$2^2 - 1 = 3$$

Suggested strategy:

- Use `np.arange` to generate the array of x values
- Compute $(2^x - 1)$ by using simple arithmetic operators like above
- For this exercise, do not use for-loops or `fromfunction()`

Note: the exponent operator in Python is two asterisks: `**`

```
In [24]: print(2**np.arange(11) - 1)
```

```
[ 0  1  3  7 15 31 63 127 255 511 1023]
```

Elementwise arithmetic between arrays

We have just seen an example of arithmetic between an array and a scalar (single value). It is also possible to perform arithmetic element-wise between two arrays:

```
In [25]: # Create a new array arr7 where each item is the product
# of the corresponding items in arr5 and arr6
arr7 = arr5 * arr6
print(arr7)
```

```
[ 0  1  4  9 16 25 36 49]
```

```
In [26]: # We can do the same with arrays of any dimension
arr8 = np.array([[1, 2], [3, 4]])
arr9 = np.array([[10, 10], [20, 20]])
print(arr8 * arr9)
```

```
[[10 20]
 [60 80]]
```

Note: in the above example, the arrays are multiplied **elementwise**; this is not a dot-product!

This is a frequent point of confusion for MATLAB users. Whereas MATLAB began life as a linear algebra system, NumPy was designed as a more general-purpose numerical computation system.

Exercise 4.4: Divide one array by a second array in an elementwise fashion and save the result as a new array. What happens if the second array contains a zero?

```
In [27]: a = np.array([1.2, 3.6, 5.3])
b = np.array([0.3, 1.5, 10.6])
```

```
### Your code here:
ratio = a / b
print('ratio =', ratio)

b[2] = 0
ratio = a / b
print('ratio =', ratio)
```

```
ratio = [4.  2.4 0.5]
ratio = [4.  2.4 inf]
```

```
/tmp/ipykernel_198124/1419267855.py:9: RuntimeWarning: divide by zero encountered in divide
  ratio = a / b
```

Some things to note here:

1. Usually when you ask python to do a division-by-zero on single values, it raises an exception (and thus stops executing the rest of your code). In contrast, a division-by-zero between numpy arrays only emits a *warning*, which allows your code to proceed anyway.
2. Inside the resulting array, you will see `inf` or `-inf` where the ratio couldn't be calculated. For other invalid operations (such as taking the square root of a negative number) you may get `nan` instead.

It is up to you to decide whether the presence of these values in your array is part of the behavior you expect or a problem to be fixed.

Exercise 4.5: Make the two arrays different sizes - how does that affect the operations above?

```
In [28]: a = np.array([1.2, 3.6])
b = np.array([0.3, 1.5, 10.6])

ratio = a / b
print('ratio =', ratio)
# this cell raises an exception!
```

ValueError

Traceback (most recent call last)

Cell In[28], line 4

```
1 a = np.array([1.2, 3.6])
2 b = np.array([0.3, 1.5, 10.6])
----> 4 ratio = a / b
      5 print('ratio =', ratio)
```

ValueError: operands could not be broadcast together with shapes (2,) (3,)

The cell above raises an exception telling us something didn't work because the shapes of the two arrays are not compatible. Take a moment to look over the message, since you will likely see errors like this in your own code. The important pieces are:

```
ValueError: operands could not be broadcast together with
shapes (2,) (3,)
```

This explains that you are trying an operation between two arrays of shape (2,) and (3,), and these shapes are not compatible for this operation

```
2 b = np.array([0.3, 1.5, 10.6])
----> 4 ratio = a / b
      5 print('ratio =', ratio)
```

This shows you exactly where in your code the error occurred.

Other mathematical functions

NumPy includes an extensive library of mathematical functions that operate on arrays. The full list can be found [here](#):

Mathematical functions

Let's look at just a couple of commonly used functions:

```
In [29]: # Generate an array containing e^x for several values of x
arr10 = np.linspace(1, 100, 7)
print(np.exp(arr10))
```

```
[2.71828183e+00 3.98247844e+07 5.83461743e+14 8.54813429e+21
1.25236317e+29 1.83480214e+36 2.68811714e+43]
```

Exercise 4.6:

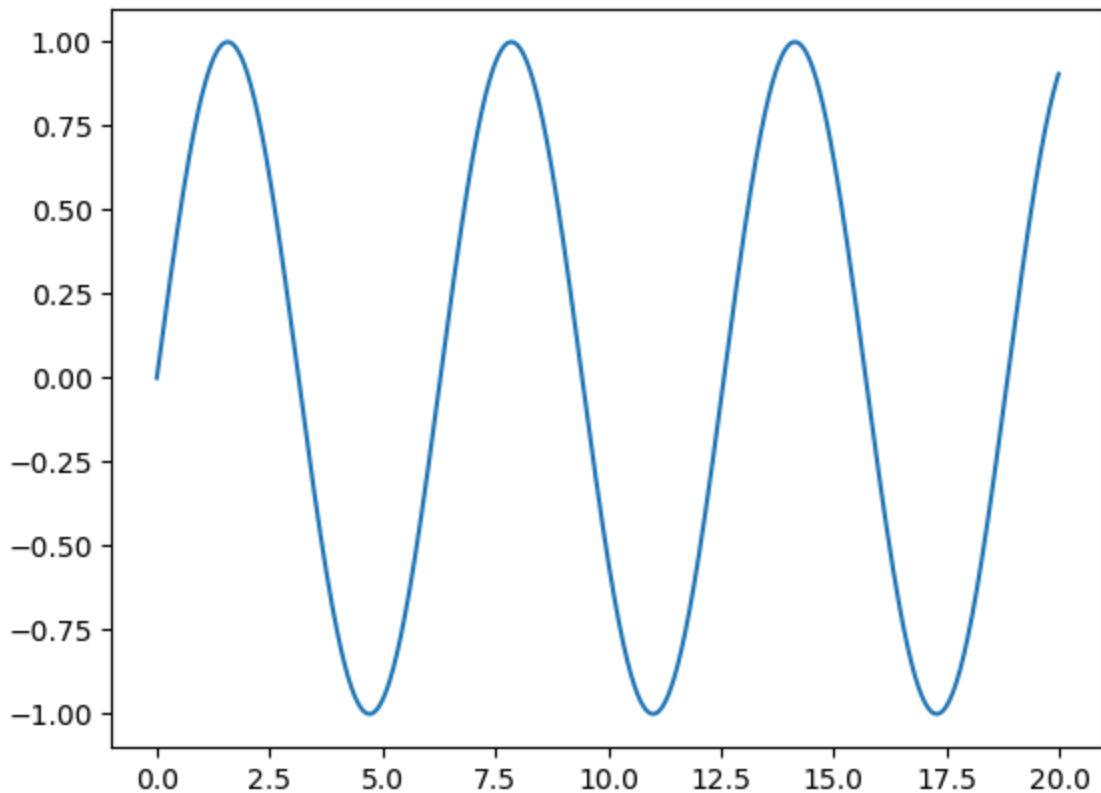
Show a plot of a sine wave.

Suggested strategy:

1. Create an array of time (or phase) values using `np.arange()` multiplied by a scale factor
2. Compute the sine of this array using `np.sin()`
3. Plot the result using `plot(x, y)`, which is imported from matplotlib below

```
In [30]: from matplotlib.pyplot import plot
```

```
### Your code here:
tvals = np.arange(1000) * 0.02
sinewave = np.sin(tvals)
plot(tvals, sinewave);
```



Accessing array sub-regions (slicing)

When dealing with arrays, it is common that we will want to select a specific sub-region of the array. This allows us to either extract a portion of the original data for focused analysis, or to modify a selected portion of the original array.

We can select a subregion of an array using indexing and slice notation, exactly as we would with a list:

```
In [31]: # To start simple, let's just take a 1D array and slice it up a few differer  
arr12 = np.arange(20)  
print(arr12)
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19]
```

```
In [32]: # Take a slice from index 5 until 15  
print(arr12[5:15])
```

```
[ 5  6  7  8  9 10 11 12 13 14]
```

```
In [33]: # Slice from the beginning until the 7th element  
print(arr12[:7])
```

```
[0 1 2 3 4 5 6]
```

```
In [34]: # Slice from the 11th element until the end  
print(arr12[11:])
```

```
[11 12 13 14 15 16 17 18 19]
```

```
In [35]: # Extract the last 5 elements
print(arr12[-5:])
```

```
[15 16 17 18 19]
```

Aside: some arrays share memory!

Note that when we slice an array, the original array is unchanged. However, the new array that we create is actually a **view** on the original array. No data is copied when the view is created. This means that modifications made to either array will be visible from both.

Multi-dimensional indexing

One major difference between lists and arrays is that we may index or slice multiple axes simultaneously:

```
In [36]: # Start with a 3D array
arr14 = np.arange(6000).reshape(40, 50, 3)

# This array has shape (40, 50, 3). Let's take a slice that contains:
# * all rows (along axis 0)
# * columns 10-19 (along axis 1)
# * only the second channel (along axis 2)
arr15 = arr14[:, 10:20, 1]
```

Note: When we **slice** an axis, the resulting array will have the same axis, but of a different length. When we **index** an axis, that axis is no longer present in the result. Put another way: **indexing** results in an array of lower dimensionality; **slicing** results in an array of the same dimensionality.

Exercise 4.7:

Try to guess what the shape of `arr15` is, then check it.

```
In [37]: print(arr15.shape)
```

```
(40, 10)
```

Slice assignment

Another major difference between lists and arrays is that, with arrays, we can actually assign new values to a slice of the array:

```
In [38]: arr16 = np.arange(18)
```

```
# Assign -1 to a block in the middle of arr16  
arr16[7:12] = -1  
print(arr16)
```

```
[ 0  1  2  3  4  5  6 -1 -1 -1 -1 -1 12 13 14 15 16 17]
```

```
In [39]: # Assign an array to fill up the middle of arr16
```

```
arr16[7:12] = np.arange(107, 112)  
print(arr16)
```

```
[ 0  1  2  3  4  5  6 107 108 109 110 111 12 13 14 15 16 17]
```

```
In [40]: # We can also do in-place arithmetic on array slices
```

```
arr16[7:12] *= -1  
print(arr16)
```

```
[ 0  1  2  3  4  5  6 -107 -108 -109 -110 -111 12 13  
 14 15 16 17]
```

Boolean operations

In addition to the arithmetic operators available with numpy arrays (+ - \/*\), most boolean (true/false) and comparison operators may be used as well. As before, these operators are applied elementwise, returning an array of boolean values:

```
In [41]: arr17 = np.arange(10)
```

```
# generate a boolean array that contains True wherever arr17 is greater than 5  
print(arr17 > 5)
```

```
[False False False False False False  True  True  True  True]
```

```
In [42]: # Combine two boolean arrays using an elementwise OR operator:
```

```
print( (arr17 < 3) | (arr17 > 7) )
```

```
[ True  True  True False False False False False  True  True]
```

Masking

NumPy provides slicing and indexing as simple ways to extract or modify a subset of your data. But what if the subset we want is more complex (not rectangular)?

The most common way of selecting an arbitrary set from an array is with boolean masking. The idea here is to create a boolean (true/false) array that has the same shape as your data, and contains the value True only in the locations of items you are interested in.

```
In [43]: # let's start with a simple array
arr18 = np.arange(10, dtype='float')

# make a mask that selects all locations where arr18 is > 6 or < 2
mask = (arr18 > 6) | (arr18 < 2)

print(arr18)
print(mask)

[0.  1.  2.  3.  4.  5.  6.  7.  8.  9.]
[ True  True False False False False False  True  True  True]
```

```
In [44]: # Now the fun part: let's see only the selected values
print(arr18[mask])

[0.  1.  7.  8.  9.]
```

```
In [45]: # We can also selectively modify these values in the original array
arr18[mask] = np.nan
print(arr18)

[nan nan  2.  3.  4.  5.  6. nan nan nan]
```

```
In [46]: # NaN (not-a-number) values appear in many kinds of data.
# One common use for boolean masking is to remove NaN values from your data
# analyzing it:
mask = np.isfinite(arr18)
print(arr18[mask])
print(arr18[mask].mean())

[2.  3.  4.  5.  6.]
4.0
```

Exercise 4.8:

In the cell below, we generate a ratio of two arrays. Use `array.mean()` to compute the average of the ratio where `input1` is less than 100.

Hint: you can use the `&` and `|` operators to combine masks together.

```
In [47]: input1 = np.array([13., 66., 54., 44., 21., 454., 12., 17.])
input2 = np.array([23., 48., 0., 21., 2., 92., 19., 30.])
ratio = input1 / input2

print(ratio)

### Your code here:
mask1 = input1 < 100
mask2 = np.isfinite(ratio)
print(ratio[mask1 & mask2].mean())
```



```
[ 0.56521739  1.375          inf  2.0952381  10.5          4.93478261
 0.63157895  0.56666667]
2.6222835167629217
```

```
/tmp/ipykernel_198124/4060846649.py:3: RuntimeWarning: divide by zero encountered in divide
ratio = input1 / input2
```

Documentation

[Boolean or "mask" index arrays](#)

Reshaping and transposing arrays

Arrays have a shape that is determined by the number of dimensions and a length along each axis:

```
In [48]: arr19 = np.arange(30)
arr19.shape
```

```
Out[48]: (30,)
```

What if we want to keep all the same values, but arrange them with a different shape? For example, the 1D array of length 30 could also be represented as a 2D array of shape (10, 3):

```
In [49]: arr20 = arr19.reshape(10, 3)
arr20
```

```
Out[49]: array([[ 0,  1,  2],
 [ 3,  4,  5],
 [ 6,  7,  8],
 [ 9, 10, 11],
 [12, 13, 14],
 [15, 16, 17],
 [18, 19, 20],
 [21, 22, 23],
 [24, 25, 26],
 [27, 28, 29]])
```

Note that when we reshape an array, the total size of the array must remain the same (in fact, the new array is just a view on the same memory).

In addition to changing the *lengths* of the array axes, we can also change their *order*:

```
In [50]: # Transpose axes such that axis 1 comes first:
arr20.transpose(1, 0)
```

```
Out[50]: array([[ 0,  3,  6,  9, 12, 15, 18, 21, 24, 27],
               [ 1,  4,  7, 10, 13, 16, 19, 22, 25, 28],
               [ 2,  5,  8, 11, 14, 17, 20, 23, 26, 29]])
```

```
In [51]: # Note that transposing is not the same as reshaping!
arr20.reshape(3, 10)
```

```
Out[51]: array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
               [10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
               [20, 21, 22, 23, 24, 25, 26, 27, 28, 29]])
```

A very common operation is to add a new axis of length 1, just to change the dimensionality of the array.

We could use `reshape()` for this, but numpy provides an easier option:

```
In [52]: arr20.reshape(10, 3, 1).shape
```

```
Out[52]: (10, 3, 1)
```

```
In [53]: arr20[:, :, np.newaxis].shape # easier because we don't need to know the ex
```

```
Out[53]: (10, 3, 1)
```

Array reduction

NumPy has many functions for performing mathematical operations on arrays. Earlier, we used **exp()** and **sin()** to create new arrays by performing these operations elementwise; that is, the output arrays had the same shape as their input arrays.

Another group of mathematical functions takes an array as input and returns either a scalar or an array with one fewer dimension than the input. This process is called **reduction**.

Reduction methods: **min()**, **max()**, **mean()**, and **std()**

The most common reductions are available as array methods (we access them by writing **data.method()** as opposed to **np.function(data)**)

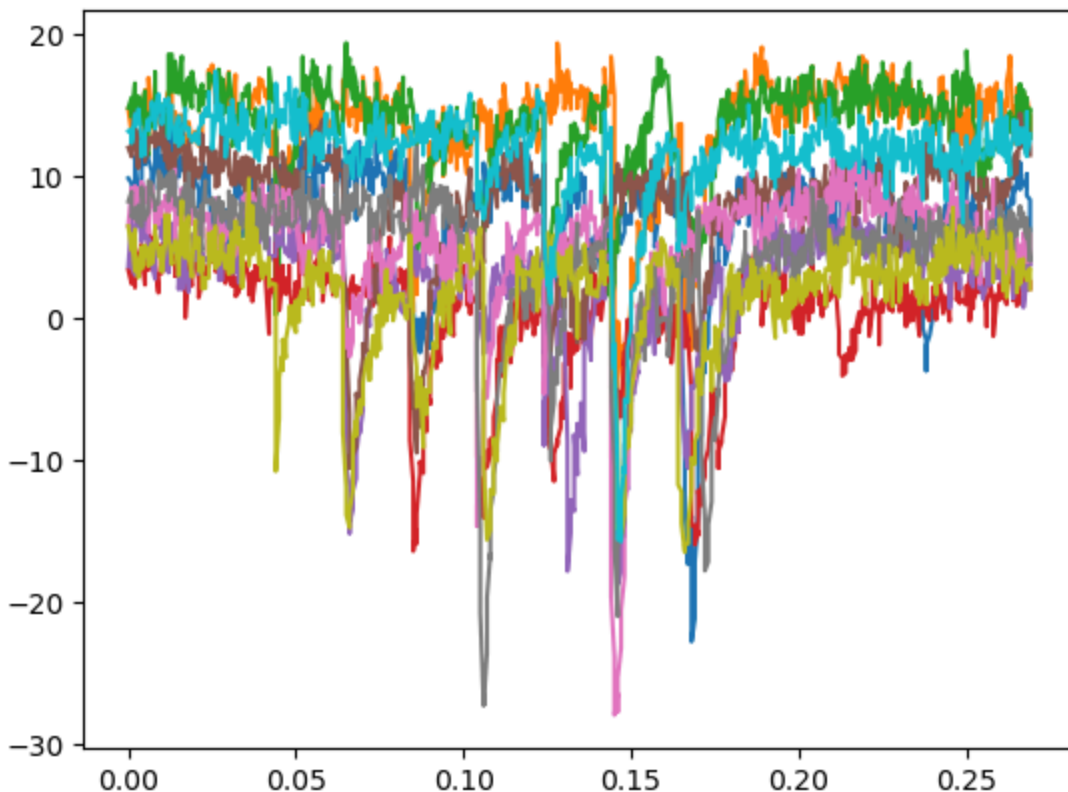
```
In [54]: # For this example, we will read a CSV file containing electrode recordings
# a patch clamp experiment.
import os
ephys_data = np.loadtxt("../support_files/datasets/sweeps.csv", delimiter=",",
print(ephys_data.shape)
```

```
(673, 11)
```

The **data** array contains 11 columns. The first column contains time values, and the rest contain electrode data recorded from 10 consecutive trials.

```
In [55]: # Separate time values from the rest of the data
tvals = ephys_data[:, 0]
ephys_data = ephys_data[:, 1:]

# Plot all sweeps overlaid
plot(tvals, ephys_data);
```



```
In [56]: # Print some statistics on the entire dataset
print("min:", ephys_data.min(), "max:", ephys_data.max())
print("mean:", ephys_data.mean(), "stdev:", ephys_data.std())
```

```
min: -27.97 max: 19.391
mean: 6.703341753343239 stdev: 6.387010694610659
```

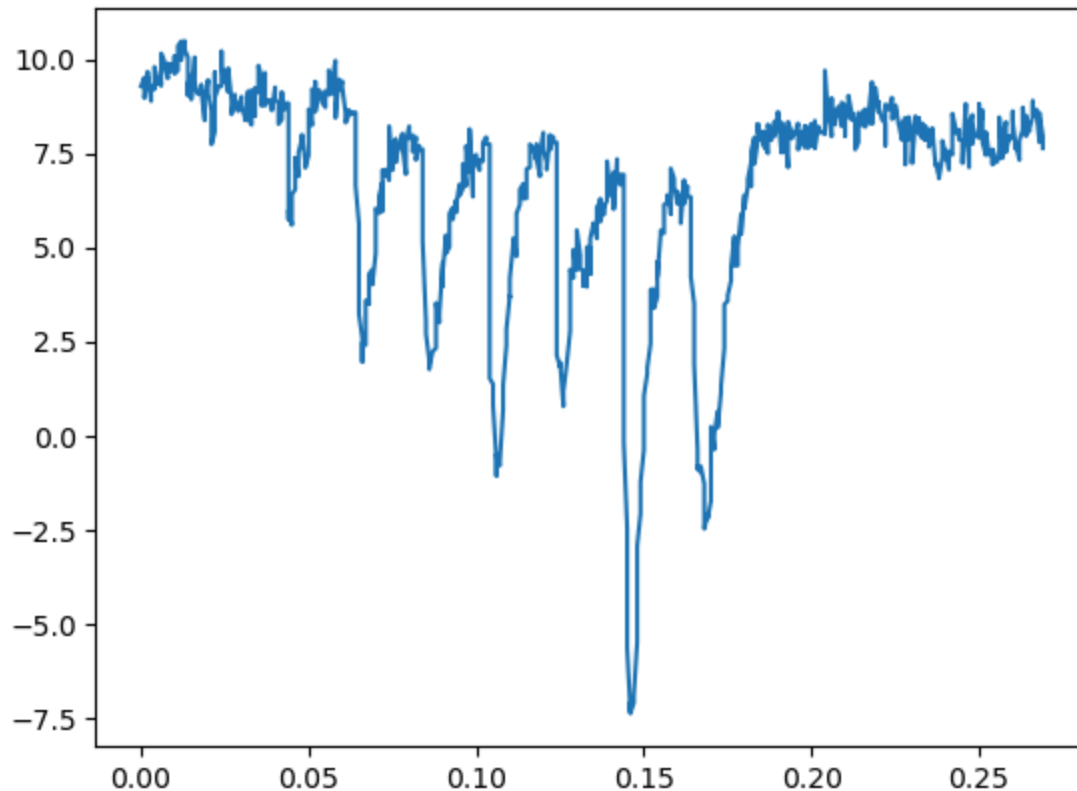
If the array is multi-dimensional, we may optionally specify an **axis** over which the reduction should be applied.

```
In [57]: # Compute the mean value from each recording
# (average all values along axis 0)
print("Mean of each recording:", ephys_data.mean(axis=0))

# For each time point, compute (and plot) the average values across all recordings
# (mean of all values along axis 1)
print("\nAverage values across all 10 recordings:")
plot(tvals, ephys_data.mean(axis=1));
```

Mean of each recording: [7.24741456 13.31140862 13.53353046 0.20036404 2.
43981724 7.96416642
5.298789 4.18689153 1.33420505 11.51683061]

Average values across all 10 recordings:



Reduction functions: **median()**, **sum()**, **argmin()**, and **argmax()**

```
In [58]: # Only the most common reductions are available as array methods.
# Many more are regular functions defined in the numpy package
# (use np.function_name() to access them)
print("Median value of all samples up to 10ms:", np.median(ephys_data[:25]))
print("Sum (area under curve) for each recording:", np.sum(ephys_data, axis=
```

Median value of all samples up to 10ms: 9.384
Sum (area under curve) for each recording: [4877.51 8958.578 9108.066 134.
845 1641.997 5359.884 3566.085 2817.778
897.92 7750.827]

```
In [59]: # Another useful set of functions gives the _index_ within
# an array where certain features occur:
print("Minimum (peak) value of each recording occurs at indices", np.argmin(
print("Minimum (peak) value of average recording occurs at t =", 0.4e-3 * np
```

Minimum (peak) value of each recording occurs at indices [420 363 109 212 36
5 164 362 263 413 366]
Minimum (peak) value of average recording occurs at t = 0.1452

```
In [60]: # Save your hard work!
np.save("ephys_data.npy", ephys_data)
```

```
# You could restore this array later on using np.load()
reloaded_data = np.load("ephys_data.npy")
if (reloaded_data == ephys_data).all():
    print("Data was saved and verified!")
```

Data was saved and verified!

MATLAB gotchas

Transitioning from MATLAB to python/numpy can be fun, but tricky. Here is a list of things to keep in mind:

- Matlab indices start at "1"; python/numpy start at "0"
- For loops, like MATLAB, are very slow in python, you should vectorize when possible.
- In python, matrix multiplication is done with ".dot", not "**"

Check out [Numpy for MATLAB Users](#) for code examples help ease the transition

Exercises

Exercise 4.9:

Create a 1-d numpy array of the first 10,000,000 integers, and call it x. Compute $x + x^2$ two ways:

1. By using "vectorized computations" on a single line
2. By using a for loop

Which takes longer? Extra credit: use `time.perf_counter()` to measure the difference.

```
In [61]: import time

start = time.perf_counter() # returns a float telling the current time in s

### Your code here:
x = np.arange(10000000)
x + x**2
stop = time.perf_counter()
print(f"Vectorized version took {stop - start:.2f} seconds")

start = time.perf_counter()
for xi in x:
    xi + xi**2
```

```
stop = time.perf_counter()
print(f"For-loop version took {stop - start:.2f} seconds")
```

Vectorized version took 0.30 seconds

For-loop version took 1.39 seconds

Exercise 4.10:

Create a 1-d numpy array (call it `ts` for timestamps), with increasing integers from 3 to 300 (including 3 but not 300) with increment of 3, and print the array

```
In [62]: ts = np.arange(3, 300, 3)
print(ts)
```

```
[ 3  6  9 12 15 18 21 24 27 30 33 36 39 42 45 48 51 54
 57 60 63 66 69 72 75 78 81 84 87 90 93 96 99 102 105 108
111 114 117 120 123 126 129 132 135 138 141 144 147 150 153 156 159 162
165 168 171 174 177 180 183 186 189 192 195 198 201 204 207 210 213 216
219 222 225 228 231 234 237 240 243 246 249 252 255 258 261 264 267 270
273 276 279 282 285 288 291 294 297]
```

Exercise 4.11:

change the dtype of `ts` into 32-bit floating point and print `ts`

```
In [63]: ts = ts.astype(np.float32)
print(ts)
```

```
[ 3.  6.  9. 12. 15. 18. 21. 24. 27. 30. 33. 36. 39. 42.
 45. 48. 51. 54. 57. 60. 63. 66. 69. 72. 75. 78. 81. 84.
 87. 90. 93. 96. 99. 102. 105. 108. 111. 114. 117. 120. 123. 126.
129. 132. 135. 138. 141. 144. 147. 150. 153. 156. 159. 162. 165. 168.
171. 174. 177. 180. 183. 186. 189. 192. 195. 198. 201. 204. 207. 210.
213. 216. 219. 222. 225. 228. 231. 234. 237. 240. 243. 246. 249. 252.
255. 258. 261. 264. 267. 270. 273. 276. 279. 282. 285. 288. 291. 294.
297.]
```

Exercise 4.12:

add 0.5 to every element in `ts` which is larger than 200 and print `ts`

```
In [64]: ts[ts > 200] += 0.5
print(ts)
```

```
[ 3.    6.    9.   12.   15.   18.   21.   24.   27.   30.   33.   36.
 39.   42.   45.   48.   51.   54.   57.   60.   63.   66.   69.   72.
 75.   78.   81.   84.   87.   90.   93.   96.   99.  102.  105.  108.
111.  114.  117.  120.  123.  126.  129.  132.  135.  138.  141.  144.
147.  150.  153.  156.  159.  162.  165.  168.  171.  174.  177.  180.
183.  186.  189.  192.  195.  198.  201.5 204.5 207.5 210.5 213.5 216.5
219.5 222.5 225.5 228.5 231.5 234.5 237.5 240.5 243.5 246.5 249.5 252.5
255.5 258.5 261.5 264.5 267.5 270.5 273.5 276.5 279.5 282.5 285.5 288.5
291.5 294.5 297.5]
```

Exercise 4.13:

calculate the mean difference between adjacent elements and print it

[hint](#)

```
In [65]: print(np.mean(np.diff(ts)))
```

```
3.0051022
```

Exercise 4.14:

find the index of the element closest to the number 212.3, and print that index and print the element

```
In [66]: ind = np.argmin(np.abs(ts-212.3))
print('index:', ind, ';', 'value:', ts[ind])
```

```
index: 70 ; value: 213.5
```

Advanced topics in NumPy

argwhere()

Whereas `argmin()` and `argmax()` return only a single index (or reduce `input.ndim` by 1), `argwhere()` returns an **array** of indices where its input is True. This provides a more powerful tool for locating features within our data.

```
In [67]: arr17 = np.array([
    [True, False, False, True, False],
    [False, False, True, False, False],
])
print(arr17)
print(np.argwhere(arr17))
```

```
[[ True False False  True False]
 [False False  True False False]]
[[0 0]
 [0 3]
 [1 2]]
```

`argwhere()` is most useful when combined with the boolean / comparison operators we saw earlier

```
In [68]: # Find indices where arange(10) is greater than 7
np.argwhere(np.arange(10) > 7)
```

```
Out[68]: array([[8],
               [9]])
```

Documentation

[Searching, sorting, and counting](#)

Exercise 4.15:

Load the image included with this notebook and use `argwhere()` to determine the cell body's center of mass.

Hint: The pixels in the soma all have a value < 50

Suggested strategy:

1. Make a boolean array that is True wherever the image value is < 50
2. Use `argwhere` to get the positions of all True values in the boolean array
3. Average pixel locations to get the center of mass

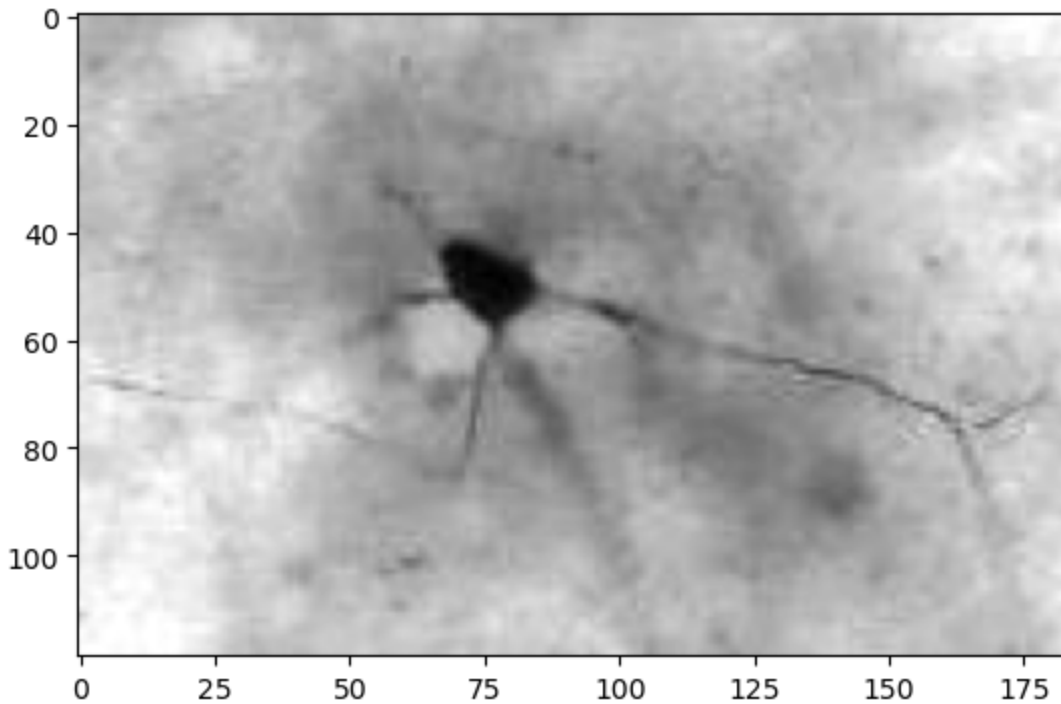
```
In [69]: from PIL import Image
import os
img = Image.open("../support_files/images/neuron.jpg")

# convert the image to an array and discard the blue/green channels
# (the image is greyscale anyway, so we don't need color)
img_data = np.array(img)[: , : , 0]

imshow(img_data, cmap='gray');

### Your code here:
soma_pixels = np.argwhere(img_data < 50)
print("Center:", soma_pixels.mean(axis=0))
```

Center: [49.27388535 75.66242038]



Exercise 4.16:

The code below creates a data set similar to a calcium imaging experiment. The array is 3D (frame, x, y) and contains one small region (a "cell") where the brightness of the image fluctuates over time.

The purpose of this exercise is to generate a plot of the signal hidden in this region.

Suggested approach:

1. Use `data.std` to measure the standard deviation of each pixel over time
2. Display the stdev image with `imshow` to visually determine the location of the cell
3. Slice out the region of the image around the cell
4. Take the average over each frame of the sliced data and plot it

```
In [70]: # Create one frame of static, low-frequency noise
from scipy.ndimage import gaussian_filter
noise = np.random.normal(size=(200, 200), loc=128, scale=40)
static = gaussian_filter(noise, 15) + 0.3 * gaussian_filter(noise, 5)

# Create full 3D array with static and time-varying noise
n_frames = 200
video_data = np.random.normal(size=(n_frames, 200, 200)) + static.reshape(1,

# Create random calcium transient signal
np.random.seed(0) # this is added to keep the problem stable in the answer
sig = np.zeros(n_frames)
spike_times = np.random.randint(10, n_frames-10, size=6)
dt = 0.1 # let's say frames are collected at 10 Hz
```

```

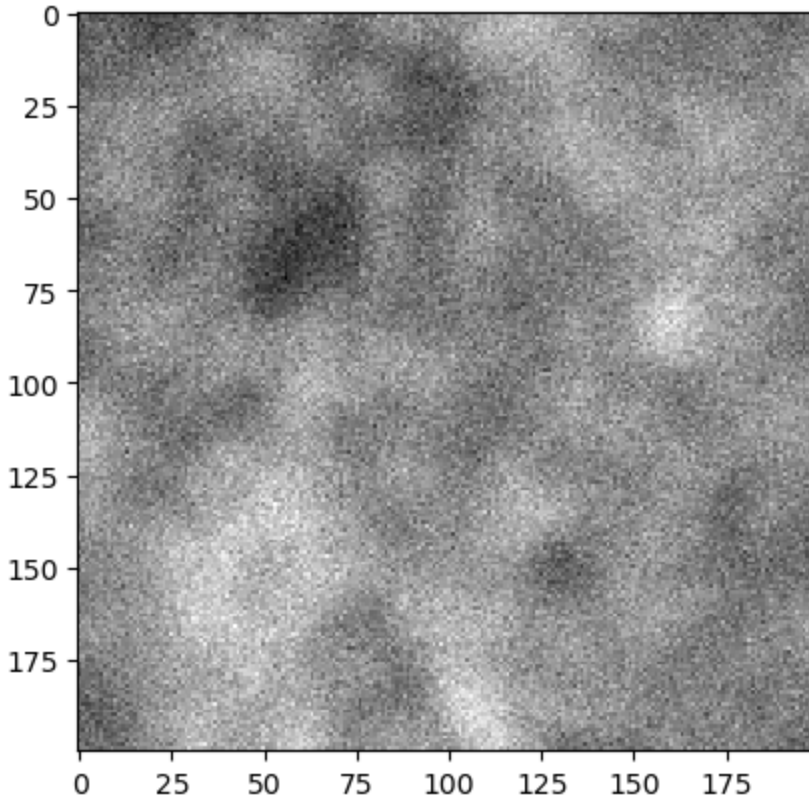
tau = 0.5 # and calcium signals have a decay time constant of 0.5 sec
for spike in spike_times:
    sig[spike:] += 1.5 * np.exp(-np.arange(n_frames-spike)*dt / tau)

# Add calcium signal to a randomly-chosen region of the image
i,j = np.random.randint(10, video_data.shape[0]-10, size=2)
video_data[:, i:i+10, j:j+10] += sig.reshape(sig.shape[0], 1, 1)

# Show the first frame.
print("This data is noisy! Where is the cell?")
imshow(video_data[0], cmap='gray', interpolation='none');

```

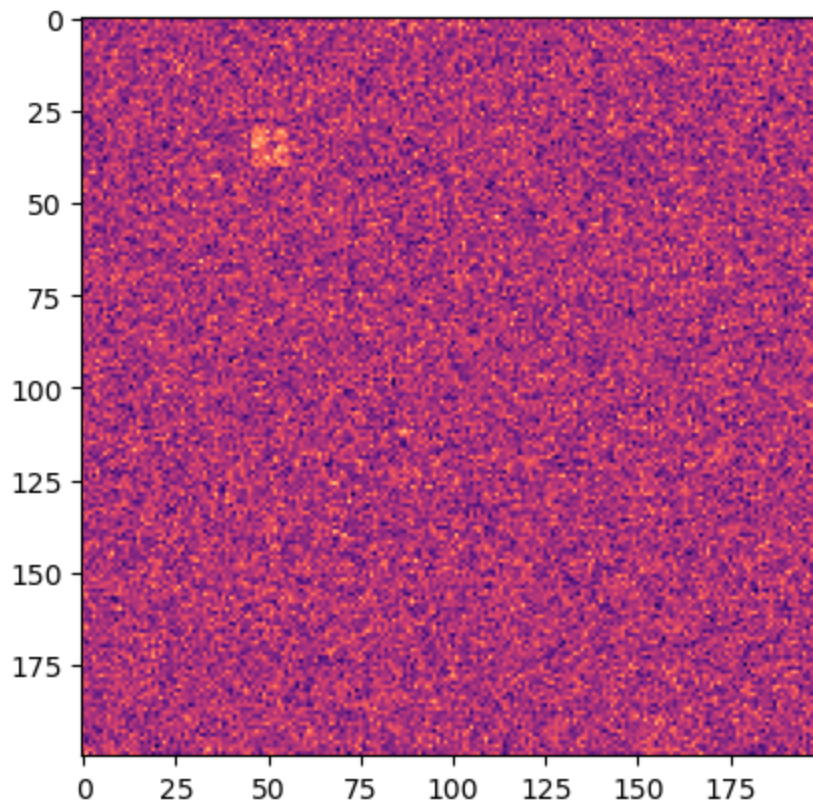
This data is noisy! Where is the cell?



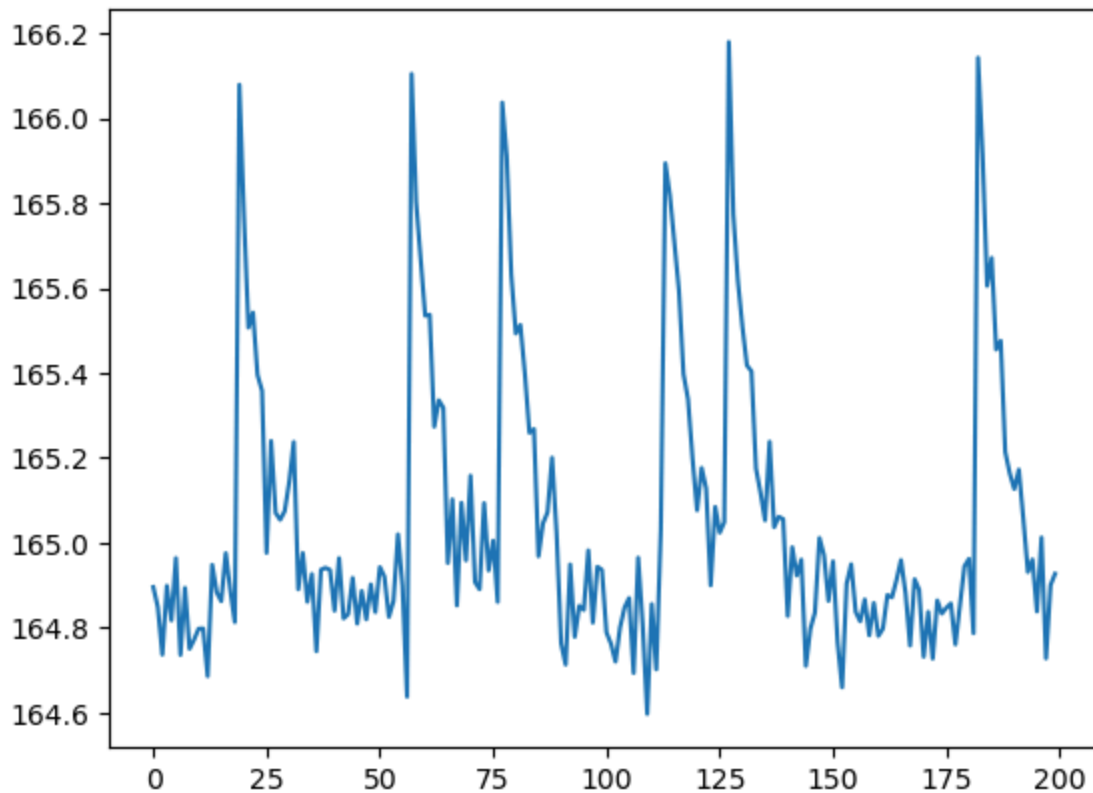
```

In [71]: # Part 1: Calculate the standard deviation of each pixel across all frames
# and display the result as an image.
### Your code here:
stdev = video_data.std(axis=0)
imshow(stdev, cmap='magma');

```



```
In [72]: # Part 2: Now that you can see the location of the cell, slice its region  
# from the video_data array and plot the average intensity over time.  
  
### Your code continued:  
# Looks like the cell is at about (30:40, 45:55)  
cell_data = video_data[:, 30:40, 45:55]  
  
# Average over x,y axes to get signal  
my_data = cell_data.mean(axis=2).mean(axis=1)  
  
plot(my_data);
```



```
In [73]: # Save your hard work!
np.save("my_data.npy", my_data)

# You could restore this array later on using np.load()
reloaded_data = np.load("my_data.npy")
if all(reloaded_data == my_data):
    print("Data was saved and verified!")
```

Data was saved and verified!

Note: `np.save()` is a quick-and-dirty solution for storing data, such as when caching intermediate results in an extended analysis. Because it uses a nonstandard format, it is not a great choice for archival storage or data sharing. In those cases, consider something like CSV (`np.savetxt`) or HDF5 (via the 3rd-party `h5py` package).

Documentation

[Input and Output h5py](#)

Fancy indexing: using an array of indices to select values from another array

Another powerful way to select items from an array is by specifying the exact set of indices you would like to extract. This is called "fancy indexing" (It's also called "array

indexing with index arrays", but I prefer "fancy"):

```
In [74]: # Again, start with a simple array:
arr19 = np.arange(100, 110)

# let's say I have an array containing the indices that I want to extract fr
indices = np.array([1,5,2,3,2,6,6,0])

# Extracting values from arr19 gives:
print(arr19[indices])

[101 105 102 103 102 106 106 100]
```

Notice that with fancy indexing, you can select values from the input array multiple times and in any order.

This feature is used, for example, to implement lookup tables.

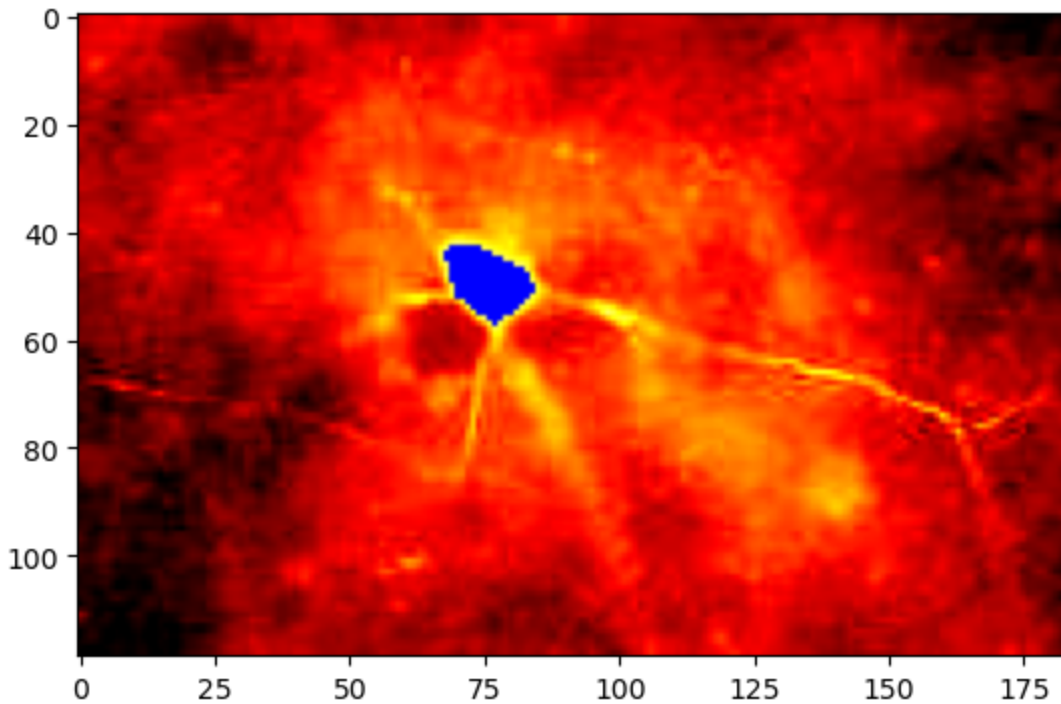
```
In [75]: # Load the neuron image again and display it with color derived from a lookup
import os
img = Image.open("../support_files/images/neuron.jpg")
img_data = np.array(img)[:,: , 0]

# Construct a lookup table of RGB color values that transition from
# white -> yellow -> red -> black
lut = np.zeros((256, 3), dtype='ubyte')
ind = np.arange(255, -1, -1)
lut[:,0] = np.clip(ind*3, 0, 255)
lut[:,1] = np.clip(ind*3-256, 0, 255)
lut[:,2] = np.clip(ind*3-512, 0, 255)

# Replace everything < 50 (just the soma in this case) with blue
lut[:50] = (0, 0, 255)

# Create an RGB-colored image by using each value in <b>img_data</b> as an i
# to look up the corresponding color in <b>lut</b>
color_data = lut[img_data]

imshow(color_data);
```



Exercise 4.17:

Calculate the average value of the pixels both inside and outside the cell body.

```
In [76]: mask = img_data < 50
print("Average value inside soma:", img_data[mask].mean())
print("Average value outside soma:", img_data[~mask].mean())
```

```
Average value inside soma: 24.96815286624204
Average value outside soma: 181.45043470260822
```

Documentation

[Index arrays](#)

Array concatenation / stacking

Let's say we have two (or more) arrays that we would like to combine together into a single array. Numpy provides several functions that accomplish this task in different ways, but let's just focus on the most general ones: **concatenate()** and **stack()**. This function takes a list of arrays and stacks them together along any axis.

The rules of concatenation are:

- All arrays must have the same number of dimensions (you can create new dimensions of length-1 if needed)
- All arrays must have the same shape **except** along the concatenation axis

```
In [77]: # Start with two arrays
arr20 = np.arange(5)
arr21 = np.arange(100, 105)

# concatenate these along axis 0
concat = np.concatenate([arr20, arr21])

print(concat)

[ 0  1  2  3  4 100 101 102 103 104]
```

```
In [78]: # now stack the arrays across a new axis
stacked = np.stack([arr20, arr21], axis=0)
print(stacked)

[[ 0  1  2  3  4]
 [100 101 102 103 104]]
```

```
In [79]: # Can I add another row to stacked ?
arr22 = np.arange(200, 205)

# Yes, but I need to make sure that my arrays have the same number of dimensions
# stacked is 2D, whereas arr22 is only 1D. I need to reshape it to become 2D
arr22_2d = arr22.reshape(1, 5)
print(np.concatenate([stacked, arr22_2d], axis=0))

[[ 0  1  2  3  4]
 [100 101 102 103 104]
 [200 201 202 203 204]]
```

Documentation

[Array manipulation routines](#)

Broadcasting

We have seen many cases where we perform an operation elementwise between two arrays. For example:

```
# add two arrays together
arr1 + arr2

# compare two arrays
arr1 > arr2

# assign from one array to another
arr1[:,0] = arr2
```

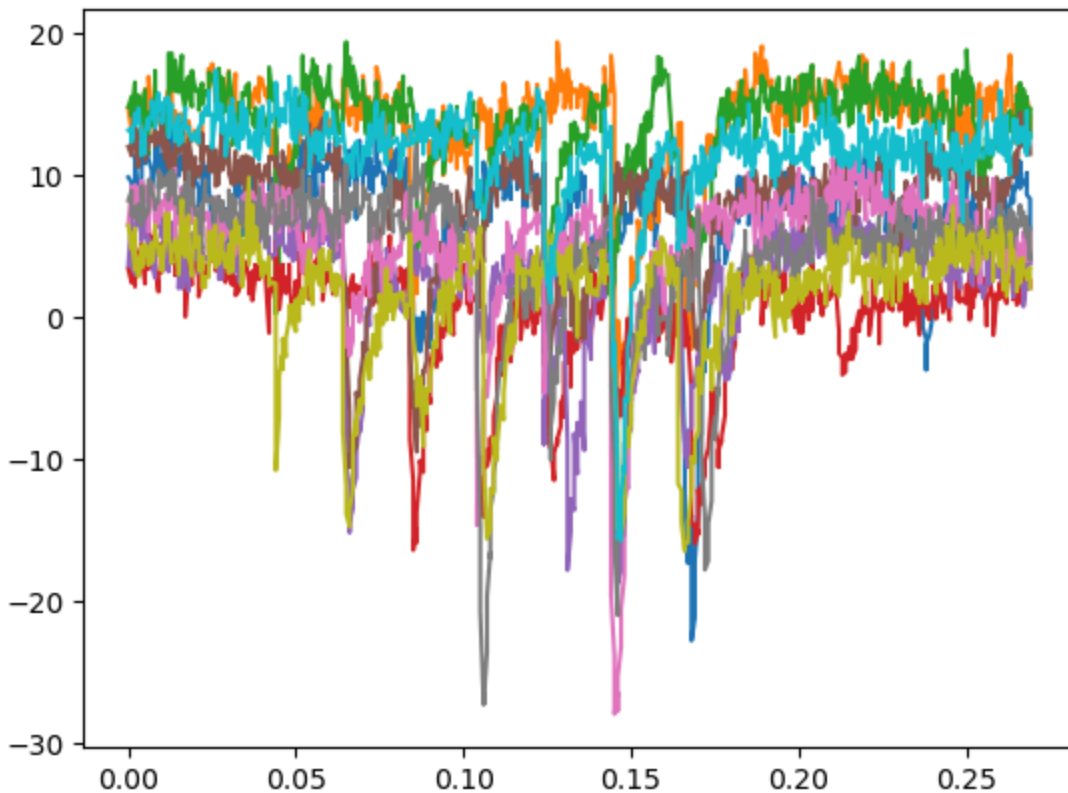
In each of these cases, the two arrays being operated on had the same shape--each element in one array had a matching element in the other array.

Broadcasting is a very powerful feature of NumPy that lets us perform operations between two arrays of different shape.

```
In [80]: # Simple example: Let's take the same electrode recording data that we
# loaded earlier and subtract off the baseline value from each recording
import os
ephys_data = np.loadtxt("../support_files/datasets/sweeps.csv", delimiter=",",
tvals = ephys_data[:,0]
ephys_data = ephys_data[:, 1:]

print("Raw data:")
plot(tvals, ephys_data);
```

Raw data:



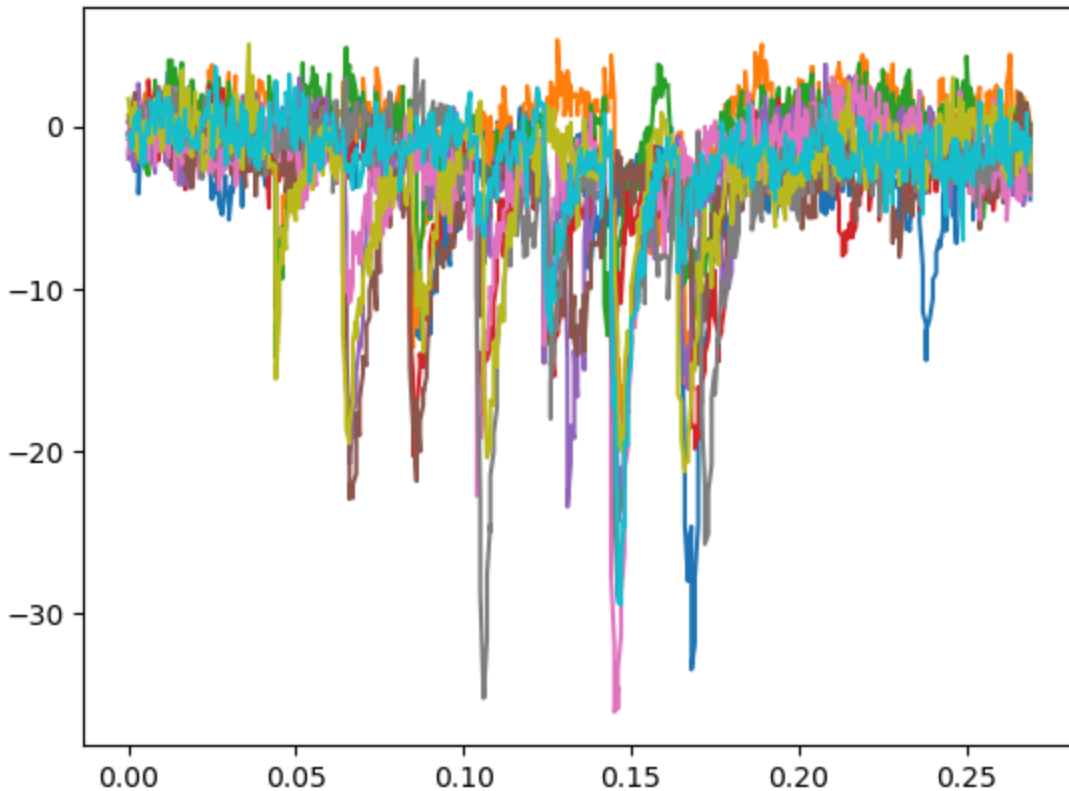
```
In [81]: # calculate the median value from the beginning (first 25 samples) of each r
base_vals = np.median(ephys_data[:25], axis=0)

# reshape base_vals so that it is clear how the axes of base_vals align with
base_vals = base_vals.reshape(1, 10)

# subtract baseline values from data
baseline_subtracted = ephys_data - base_vals

print("Baseline-subtracted data:")
plot(tvals, baseline_subtracted);
```

Baseline-subtracted data:



Notice that we had to reshape **base_vals** to become 2D before it could be subtracted from **ephys_data**. When **base_vals** is broadcast onto **ephys_data**, its first axis (which has length 1) is effectively stretched to the full length of **ephys_data**'s first axis (which has length 673).

The major rules here are:

- The arrays should have the same number of dimensions (this is not required, but it gets confusing if they are different)
- For any dimension where the array shapes are not equal, one array must have length=1 along that axis.

Documentation

[Broadcasting](#)

Exercise 4.18:

Given a set of 3D positions X (in a 3×10 array) and a 3×3 transformation matrix M , perform a matrix multiplication $M \cdot X$ using broadcasting to generate the set of transformed positions.

```
In [82]: M = np.array([
    [0, 1, 0],
    [0, 0, 1],
    [1, 0, 0],
    ])
X = np.array([
    [1, 4, 0, 8, 9, 1, 4, 0, 8, 9],
    [2, 5, 1, 7, 8, 2, 5, 1, 7, 8],
    [3, 6, 2, 6, 7, 3, 6, 2, 6, 7]
    ])
```

```
In [83]: ### Solution:
X1 = (M.reshape(3, 3, 1) * X.reshape(1, 3, 10)).sum(axis=1)
print(X1)
```

```
[[2 5 1 7 8 2 5 1 7 8]
 [3 6 2 6 7 3 6 2 6 7]
 [1 4 0 8 9 1 4 0 8 9]]
```

```
In [84]: # Test your answer against numpy's built-in matrix multiplication:
if np.all(X1 == np.dot(M, X)):
    print("We feel super smart!")
```

We feel super smart!